

00 · 总体架构设计

2026-07-31

Contents

00 · 总体架构设计	1
1. 项目背景	1
1.1 问题定义	1
1.2 为什么传统 VLM 不适合几何题	2
1.3 为什么需要 Geometry World Model	2
1.4 设计原则与权衡	2
2. 总体架构	3
2.1 五层架构	3
2.2 系统数据流图	3
2.3 模块划分	4
2.4 Agent 拓扑: Hub-and-Spoke + 流水线	5
2.5 关键数据契约	5
2.6 端到端时序	6
3. 与端到端范式的对比	6
4. 跨切关注点	6
4.1 配置管理	6
4.2 日志与可观测	7
4.3 错误处理	7
4.4 扩展点	7
5. 小结	7

00 · 总体架构设计

本文档阐述 Geometry Agent 的项目背景、传统 VLM 在几何题上的失效机理、Geometry World Model 的必要性，以及系统总体架构、模块划分、数据流与接口契约。

1. 项目背景

1.1 问题定义

本项目的核心目标是：给定一张包含几何图形的试卷图片（典型为中国中考、高考数学几何题），以及题目文字描述，系统输出完整的解答过程、数学证明过程与最终答案。形式化为：

$$\text{Solve} : (\text{Image}, \text{ProblemText}) \rightarrow (\text{Solution}, \text{Proof}, \text{Answer})$$

Image 含几何图形（印刷体或手绘体），ProblemText 含已知条件与求证/求解目标。题型涵盖：三角形、全等/相似、平行线、圆与切线、椭圆与二次曲线、动态几何等。

1.2 为什么传统 VLM 不适合几何题

当前主流 VLM（GPT-4V、Gemini、Qwen-VL、InternVL 等）在”看图说话”类任务上表现优异，但在几何题求解上存在**系统性缺陷**。这些缺陷源于几何理解任务与普通图像理解任务在**信息结构**上的本质差异。

1.2.1 几何理解与普通图像理解的本质区别

维度	普通图像理解	几何图形理解
信息载体	像素纹理、颜色、语义	精确拓扑关系、几何度量
容错性	模糊可接受（“一只猫”）	严格二值（点在线上/不在线上）
关键属性	语义类别、场景	点、线、圆的 精确位置与关系
误差传播	局部，可被上下文稀释	全局，单点错位导致整题崩溃
推理依赖	视觉先验为主	强依赖符号化、可计算关系
评判标准	IoU / 准确率	数学严格性（证明可验证）

1.2.2 VLM 失效的数学机理 VLM 的视觉编码器以**语义相似度**为目标训练，其特征空间对”语义类别”敏感，对”亚像素级几何位置”不敏感。而几何题的关键关系（如点是否在圆上）在像素层面往往只差 1~2 像素，但在语义上是二值的、决定性的。设点 P 到圆 O （圆心 O_c 、半径 r ）的距离为 $d = |P - O_c|$ 。判定 P 是否在圆上要求 $|d - r| < \epsilon$ 。当 ϵ 约为图像对角线的 1.5% 时，对应像素层面的容差仅 3~5 px。VLM 的视觉 token 分辨率（通常每 token 覆盖 16~32 px）远超此精度，故其判断本质上是”猜测”而非”测量”。

更严重的是**误差级联**：设感知层错误概率为 p_e ，证明链有 k 步，每步依赖前一步。在最坏情况下最终错误率 $P_{err} \approx 1 - (1 - p_e)^k$ 。当 $p_e = 0.2, k = 5$ 时 $P_{err} \approx 0.67$ 。这正是 VLM 端到端范式在**多步几何题**上崩溃的根源。

1.2.3 VLM 在几何题上的四类典型失败

- 1. **几何元素识别不稳定**：无法稳定识别点、线段、射线、圆、弧、角标、等长标记、平行标记等细粒度元素，同一图形多次推理结果不一致。
- 2. **拓扑关系判断错误**：对”点是否在线段上””三点共线””两线垂直/平行””点在圆上””线为切线”等关系判断准确率低。
- 3. **误差级联放大**：视觉理解一旦出错，后续 LLM 推理在错误前提下”自信地”编造出看似合理的错误证明 (hallucinated reasoning)。
- 4. **后训练成本高**：直接对大规模 VLM 做几何后训练需大量高质量标注与数百卡时 GPU，团队不具备。

1.3 为什么需要 Geometry World Model

几何题求解的根本难点不在于”推理”本身（LLM 在给定正确几何事实后，证明能力已相当强），而在于**构建一个准确、结构化、可计算的几何世界模型（Geometry World Model）**——即从图像中还原出题目所给的全部几何对象及其精确关系，并以机器可验证的形式表示出来。

此思路借鉴神经符号混合推理（Neuro-Symbolic Reasoning）：神经网络负责感知（Perception），符号系统负责推理与验证（Reasoning & Verification），二者通过结构化中间表示（Geometry Graph + DSL）解耦。

因此，本系统**刻意避免** 图片 → LLM → 答案的端到端范式，转而采用**多阶段、可解释、可验证**的流水线架构，将”看图”与”推理”彻底分离，并在两者之间引入显式的几何知识图谱与约束验证机制。

1.4 设计原则与权衡

原则	含义	代价/权衡
不依赖大规模后训练	充分利用开源模型 + 传统 CV + 符号求解	流水线更长，工程更复杂
高准确率优先	感知层宁可召回不足也不要错误关系	可能漏检导致解题失败（用多假设缓解）
可解释	每步输出可追溯	需维护 verification_log
可验证	所有关系由 Verifier 判定真伪	验证器本身需正确（用合成数据测试）
可扩展	新对象/关系可独立加入	需统一接口规范
低成本数据	程序化合成 + 少量人工	合成 → 真实的域差距需风格增强

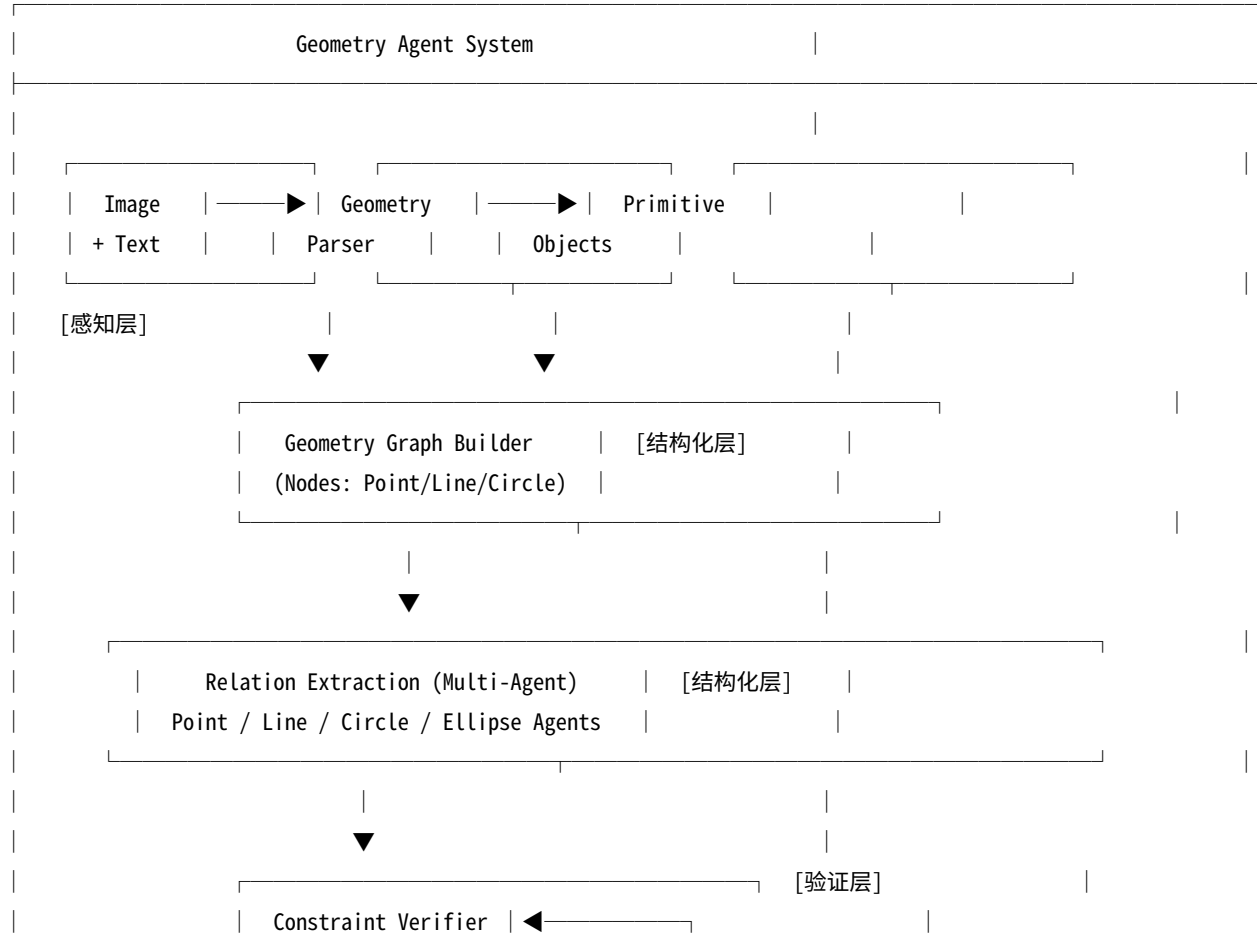
2. 总体架构

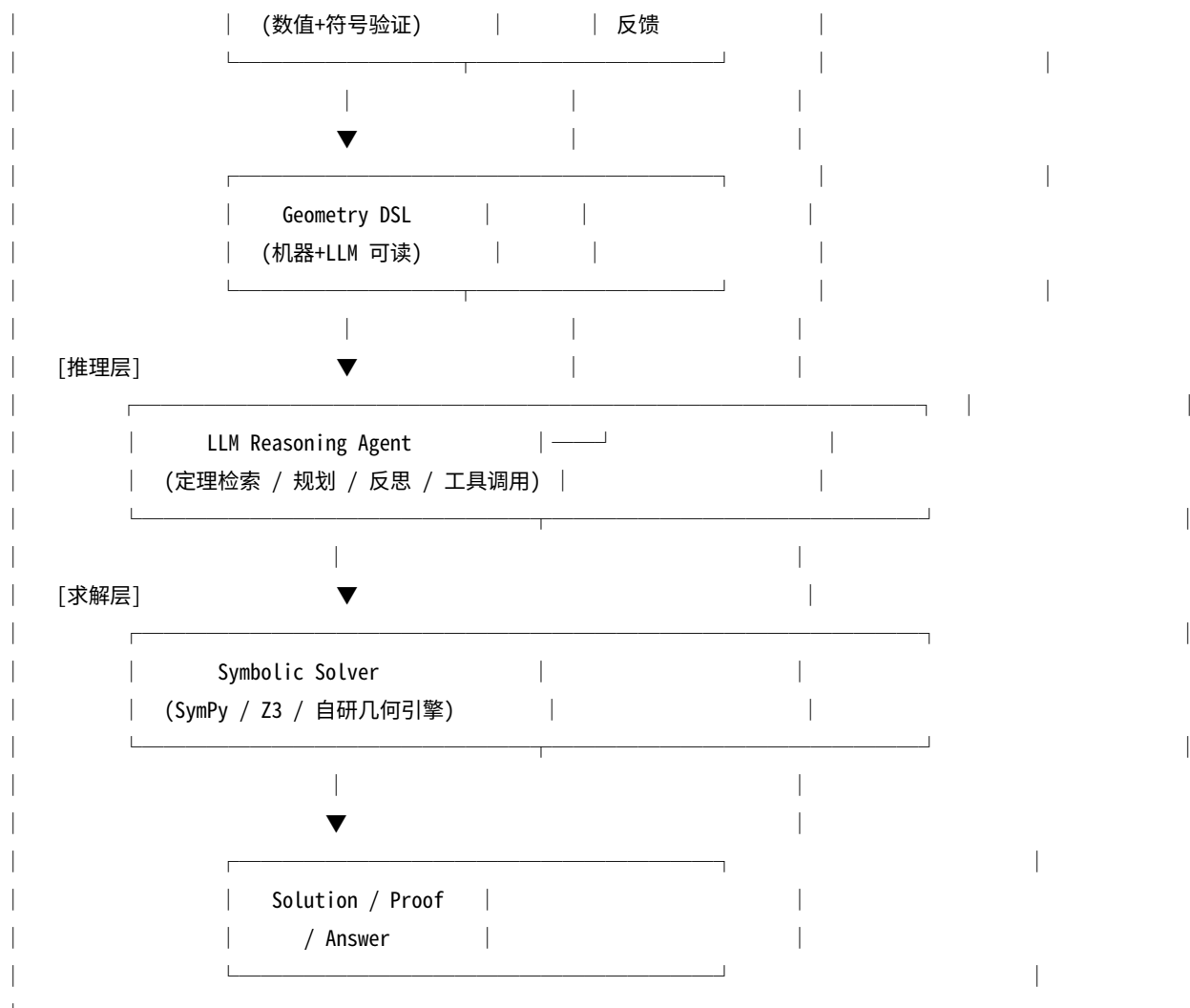
2.1 五层架构

系统采用分层、多 Agent、神经符号混合架构，划分为五层：

1. **感知层 (Perception Layer)**：图像 → 几何原语（点、线、圆、椭圆、标记）。
2. **结构化层 (Structuring Layer)**：原语 → Geometry Graph，由多 Agent 抽取关系。
3. **验证层 (Verification Layer)**：关系数值/符号验证，过滤错误假设。
4. **推理层 (Reasoning Layer)**：LLM 基于 Graph + DSL 进行定理搜索与证明规划。
5. **求解层 (Solving Layer)**：Symbolic Solver 执行精确计算与证明，输出最终答案。

2.2 系统数据流图





2.3 模块划分

层	模块	职责	主要技术	详见
感知层	Geometry Parser	图像 → 几何原语	OpenCV、SAM、YOLO/DETR	01, 02
结构化层	Geometry Graph Builder	原语 → 知识图谱	NetworkX、图构建	03
结构化层	Relation Extraction Agents	关系抽取	Multi-Agent、规则+LLM	04
验证层	Constraint Verifier	关系真伪判定	数值计算、容差模型	05
推理层	Geometry DSL	统一中间表示	YAML/JSON DSL	06
推理层	LLM Reasoning Agent	证明规划	CoT、Tree Search、Reflection	07
求解层	Symbolic Solver	精确计算与证明	SymPy、Z3、自研引擎	08
数据层	Dataset & RAG	知识库与样本	向量库、定理库	09

2.4 Agent 拓扑: Hub-and-Spoke + 流水线

- **Perception Spoke**: Point/Line/Circle/Ellipse/Mark Detector 并行感知, 由 Geometry Parser 统一调度。
- **Relation Spoke**: Point/Line/Circle/Ellipse/Polygon/Mark/Cross Agent 并行抽取关系, 写入 Graph。
- **Hub**: LLM Reasoning Agent 作为中枢, 协调 Verifier 与 Symbolic Solver, 形成“假设—验证—求解”闭环。
- **反馈回路**: Verifier 对 LLM 提出的几何假设返回真/假/不确定; Solver 对计算步骤返回精确结果; LLM 据此修正规划 (Self-Reflection)。

2.5 关键数据契约

系统各模块间通过明确的数据契约解耦。以下为贯穿全流水线的核心数据结构 (详见各子文档)。

```
{
  "image_path": "data/exam/2023_gaokao_q5.png",
  "problem_text": "如图, 在  $\triangle ABC$  中, D 为 BC 中点, AD 的延长线交  $\triangle ABC$  的外接圆于 E。求证:  $AB \cdot AC = AD \cdot AE$ 。",
  "metadata": {"source": "2023 高考", "grade": "高中", "type": "证明题"}
}
```

2.5.1 输入

```
{
  "points": [{"id": "P_A", "label": "A", "coords": [180.0, 84.5], "confidence": 0.97}],
  "segments": [{"id": "L_AB", "label": "AB", "endpoints": [[180, 84.5], [260, 140]], "equation": {"a": 0.62, "b": -0.78, "c": -8.4}},
  "circles": [{"id": "C_0", "label": "O", "center": [180, 160], "radius": 75.5}],
  "ellipses": [],
  "marks": [{"id": "M_1", "type": "right_angle", "vertex": "P_A"}]
}
```

2.5.2 PrimitiveSet (感知层输出 → 结构化层输入)

```
{
  "graph_version": "1.0",
  "nodes": [{"id": "P_A", "type": "Point", "label": "A", "coords": [180, 84.5]}],
  "edges": [{"src": "P_A", "dst": "C_0", "rel": "On", "confidence": 0.98, "verified": true}]
}
```

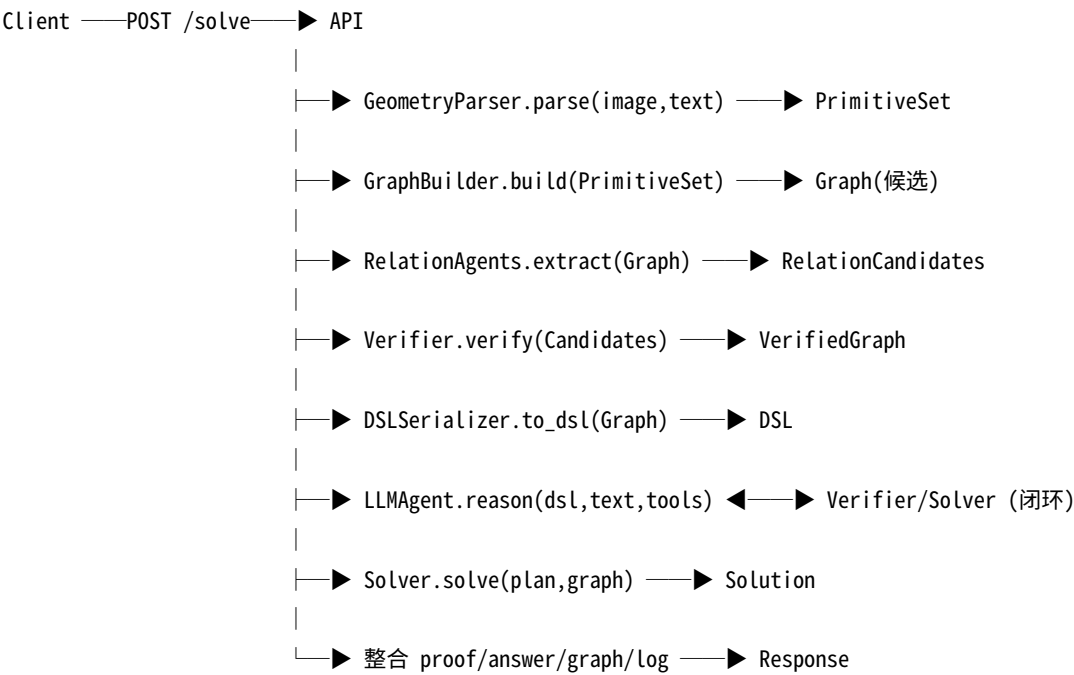
2.5.3 GeometryGraph (结构化层输出 → 验证层/推理层)

```
{
  "answer": "AB · AC = AD · AE 成立",
  "proof": [
    {"step": 1, "statement": "A, B, C, E 共圆", "reason": "已知"},
    {"step": 2, "statement": " $\angle ABE = \angle ACE$ ", "reason": "同弧圆周角"}
  ],
  "geometry_graph": "<Geometry Graph JSON>",
}
```

```
"confidence":0.93,
"verification_log": "< 验证器执行日志 >"
}
```

2.5.4 最终输出

2.6 端到端时序



3. 与端到端范式的对比

维度	端到端 VLM	本系统
感知精度	像素级，模糊	符号级，精确（亚像素拟合）
关系判定	隐式猜测	显式验证（数值 + 符号）
误差传播	全局级联	层间隔离，验证器阻断
可解释	黑盒	每步可追溯
可验证	否	是（Solver/Lean）
训练成本	高（大规模后训练）	低（仅小模型微调）
延迟	低	较高（多阶段）

本系统以**更高的延迟与工程复杂度**换取**准确性、可解释性、可验证性**，这在数学证明场景下是合理且必要的权衡。

4. 跨切关注点

4.1 配置管理

所有阈值、模型路径、LLM 参数集中于 configs/*.yaml，由 Pydantic 校验。

4.2 日志与可观测

每个模块输出结构化日志 (JSON Lines), 含 module/step/input_hash/output/confidence/duration。verification_log 全程累积, 支持回放调试。

4.3 错误处理

采用**多假设保留 + 降级**策略: 歧义图元保留候选; 检测失败降级为传统 CV; LLM 失败触发 Reflection; Solver 失败返回”无法证明”而非编造。

4.4 扩展点

- 新几何对象: 新增 Detector + Node 类型 + Agent。
 - 新关系: 新增 Edge 类型 + Verifier。
 - 新定理: 定理库新增条目, RAG 自动检索。
-

5. 小结

本架构的核心创新在于: **用 Geometry World Model (Geometry Graph + DSL + Verifier) 将感知误差隔离在推理之外**, 使 LLM 在”已验证的符号化几何事实”上发挥推理能力, 并通过 Symbolic Solver 保证数学严格性。后续各子文档将逐一展开每个模块的设计路线。